HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING

FACULTY OF INTERNATIONAL EDUCATION

PROJECT INTERNET OF THINGS GROUP 7

COLD-CHAIN MONITORING AND INTELLIGENT STORAGE MANAGEMENT

TRẦN VẠN PHƯỚC **ID**: **24161126**

LƯƠNG THẾ QUỐC **ID**: **24161127**

NGUYỄN VĂN QUÝ **ID**: **24161128**

TRỊNH NGỌC TÍN **ID**: **24161140**

MAJOR: **ELECTRICAL AND ELECTRONIC ENGINEERING TECHNOLOGY**

ADVISOR: **HUYNH THE THIEN, PHD.**

HO CHI MINH CITY, OCTOBER 7TH 2026

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY AND ENGINEERING

FACULTY OF INTERNATIONAL EDUCATION

PROJECT INTERNET OF THINGS GROUP 7

COLD-CHAIN MONITORING AND INTELLIGENT STORAGE MANAGEMENT

TRẦN VẠN PHƯỚC **ID**: **24161126**

LƯƠNG THẾ QUỐC **ID**: **24161127**

NGUYỄN VĂN QUÝ **ID**: **24161128**

TRỊNH NGỌC TÍN **ID**: **24161140**

MAJOR: **ELECTRICAL AND ELECTRONIC ENGINEERING TECHNOLOGY**

ADVISOR: **HUYNH THE THIEN, PHD.**

HO CHI MINH CITY, OCTOBER 7TH 2026

TABLE OF CONTENTS

INTRODUCTION 1

1. Abstract 1

2. Research Objectives 1

CHAPTER 1: OVERVIEW AND PROBLEM ANALYSIS 2

1.1 Cold-Chain Context and Problem Statement 2

1.2 Scope and Limitations 3

1.3 System Requirements and Evaluation Criteria 4

CHAPTER 2: THEORETICAL BASIS AND HARDWARE SELECTION 6

2.1 Theoretical Basis of Cold Chain Monitoring 6

2.1.1 Impact of Temperature and Humidity on Sensitive Commodities 6

2.2 Hardware Component Selection and Analysis 8

2.2.1 Central Microcontroller ESP32 DevKit V1 Integrated ESP32-WROOM-32 module 8

2.2.2 DHT22 Air Temperature and Humidity Measurement Sensor 9

2.2.3 DS18B20 Accurate Core Temperature Sensor 1-Wire Protocol 10

2.2.4 28BYJ-48 Stepper Motor and ULN2003 IC 10

2.2.5 Isolated Cooling Control Mechanism 5V Relay Module 12

2.3 Software Technology Basis 12

2.3.1 MQTT Communication Protocol and EMQX Broker 12

2.3.2 Node.js Execution Environment and SQLite Database 13

2.3.3 WebSockets Technology and Chart.js Visualization 13

CHAPTER 3: SYSTEM ARCHITECTURE DESIGN 14

3.1 Overall System Architecture 14

3.2 Data Flow Diagram DFD 15

3.3 Alert and Exception Handling Flow 16

3.3.1 Warehouse Door Open/Close Event Flow Door Event Flow 16

3.3.2 Network Disconnection and Compensatory Synchronization Flow Network Failure & Sync Recovery 16

CHAPTER 4: HARDWARE AND SOFTWARE DESIGN AT THE BORDER – ESP32 18
4.1 Hardware Design and Wiring Diagram Hardware Architecture & Wiring Diagram 18
4.1.1 Overall Edge Node Hardware Architecture 18
4.1.2 GPIO Pin Allocation and Planning for ESP32 Microcontroller 19
4.1.3 Detailed Technical Wiring Analysis for Each Hardware Module 19
4.1.4 Sensor Calibration Procedure with Reference Equipment 20
4.2 ESP32 Firmware Software Algorithm Source Code Analysis 21
4.2.1 Exponential Moving Average EMA Noise Filtering Algorithm 21
4.2.2 Thermal Finite State Machine Thermal FSM & Thermal Exposure Algorithm 21
4.2.3 Non-blocking Stepper Motor Control 22
CHAPTER 5: BACKEND, DATABASE, AND API COMMUNICATION 26
5.1 Database Design 26
5.2 MQTT Ingestion and Data Processing 27
5.3 REST API Communication and Response Structure 28
CHAPTER 6: DESIGN OF INTELLIGENT MONITORING DASHBOARD 30
6.1 User Interface Design (UI/UX) 30
6.2 Real-time Connection via MQTT/WebSockets and Alert Logic 31
6.3 Real-time Data Visualization (Chart.js) 32
6.4 Integration of Historical REST API and CSV Report Export Functionality 33
CHAPTER 7: EXPERIMENTAL RESULTS AND EVALUATION 34
7.1 Scenario 1: Normal Operation and Storage Door Control 34
7.2 Scenario 2: Temperature Threshold Violation and Cooling System Response 35
7.3 Scenario 3: Disconnection and Data Backfill Recovery Mechanism 36
7.4 Overall Evaluation 37
CHAPTER 8: CONCLUSION AND FUTURE WORK 38
8.1 Comprehensive Summary of Achievements 38
8.2 Scientific and Practical Contributions 39
8.3 Technical Limitations 40

8.4 Innovative Future Directions 41

REFERENCES 43

INTRODUCTION

1. Abstract

The Cold Chain Monitoring and Smart Storage Management project focuses on researching and developing a comprehensive Internet of Things system to control storage conditions for temperature-sensitive goods such as fresh food, pharmaceuticals, and vaccines. The core differentiator of the project lies in overcoming the drawbacks of traditional warning systems by integrating a real-time thermal exposure calculation algorithm. The system utilizes an ESP32 microcontroller as an edge device to collect data, encapsulate timestamps, and transmit via the MQTT protocol. Notably, the project successfully implements an offline buffering and automatic synchronization mechanism, ensuring the integrity of the historical data chain even during network disruptions.

2. Research Objectives

The core objective of this project is to build an automated monitoring model capable of analyzing and managing storage environment risks. First, the project aims to continuously measure and monitor temperature, humidity, and storage door status in real-time to provide accurate input data. Second, the research team implements a violation assessment algorithm based on the combination of temperature deviation amplitude and duration duration-aware exposure, thereby clearly distinguishing between short-term disturbances and severe violations. Third, the system is designed with a local buffering mechanism at the edge device to preserve data during network disconnections and automatically synchronize with the server backend when the connection is restored. Finally, the project develops a centralized monitoring interface web dashboard allowing administrators to visualize historical data, track compliance status, and receive multi-level alerts.

CHAPTER 1: OVERVIEW AND PROBLEM ANALYSIS

1.1 Cold-Chain Context and Problem Statement

The Cold Chain is a complex logistics system designed to strictly maintain and control storage conditions throughout the storage, transportation, and distribution of temperature-sensitive products. For specific goods such as vaccines, biopharmaceuticals, or fresh food, maintaining a stable storage environment especially temperature and humidity is the decisive factor in preventing quality degradation and ensuring absolute safety for consumers. In practical operations, storage quality depends not only on environmental conditions at a specific moment but also on the variation of these conditions over time.

The storage environment temperature is never static and is always affected by operational disturbances. Factors such as opening warehouse doors for loading and unloading goods, changes in product load, or the automatic defrost cycles of cooling systems can all cause temporary temperature fluctuations. However, most current monitoring systems still operate based on an Instantaneous Threshold mechanism. According to this principle, as soon as the temperature parameter exceeds the allowable limit e.g., >8.0°C, the system will immediately trigger an alarm. This method reveals a core weakness: the inability to distinguish between a short-term disturbance and a prolonged violation event. Goods always possess thermal inertia; a hot air draft entering the warehouse for 30 seconds can cause the air temperature sensor to spike, but this is not enough time to alter the core temperature of the product. Continuously issuing alerts for these harmless fluctuations leads to false alarms, causing operators to become complacent and ignore genuinely critical incidents.

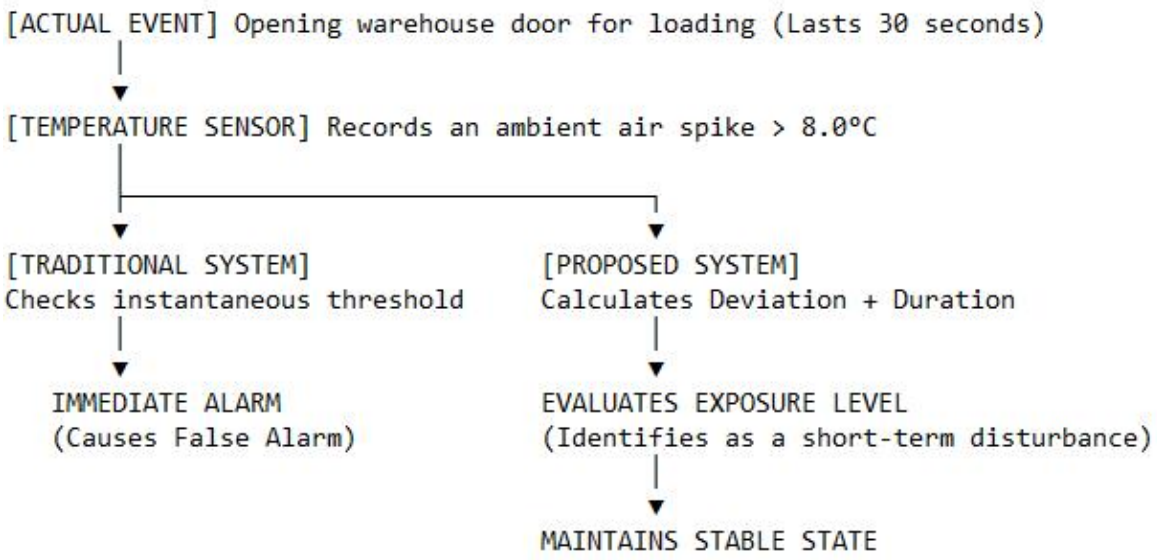


Figure 1.1: Analysis diagram of the shortcomings of the Instantaneous Threshold alarm mechanism

To thoroughly resolve this issue, the project applies the Degree-Time theoretical framework to quantify Thermal Exposure. Accordingly, the severity of a cold chain incident is not dictated solely by the amplitude of the temperature deviation, but is also heavily influenced by the cumulative duration the product is exposed to adverse conditions. The mathematical model for assessing cumulative thermal exposure is expressed by the following general formula:

$$Exposure = \sum_{i=1}^{N} T_i t_i$$

Where Ti represents the temperature deviation amplitude exceeding the threshold at the i-th sampling instance, and ti denotes the duration of the sampling cycle. By integrating this algorithm into the central processing unit, the system can completely filter out false alarms caused by brief door openings, while accurately classifying alert levels based on the actual thermal exposure of the commodities.

1.2 Scope and Limitations

The project is scoped to the development of a small-scale cold chain monitoring prototype, yet it integrates a complete End-to-End architecture from the Edge device to the Cloud computing platform and user application. The rationale for this limitation is to concentrate on validating the feasibility of the monitoring algorithm and the reliability of the data pipeline, rather than deploying it in a large-scale industrial warehouse setting.

In terms of hardware, the system utilizes the ESP32 microcontroller as the central unit for data acquisition and edge processing. The DHT22 sensor module is selected to concurrently measure temperature and humidity parameters, while the physical state of the storage door is monitored via a magnetic sensor or a slide switch. To simulate intentional environmental disturbances during the experimental phase, the research team designed a simulated door mechanism employing a 28BYJ-48 stepper motor combined with a ULN2003 driver IC, alongside a cooling system isolated and controlled via a low-voltage Relay module.

Regarding network infrastructure and software, the project restricts data transmission to the MQTT protocol utilizing a public EMQX Broker intermediary, ensuring real-time communication capabilities with optimized bandwidth. The Backend server tier is entirely programmed in a Node.js environment integrated with the Express framework, tasked with receiving JSON payloads, verifying timestamps, and executing the thermal exposure algorithm. Time-series data and event logs are stored locally on an embedded SQLite database management system, which is appropriate for the scale of a prototype model. Finally, the user interface is developed as a Web Dashboard utilizing the Tailwind CSS library for layout and Chart.js for dynamic data visualization, establishing a direct connection with the Backend via REST API standards and WebSockets.

1.3 System Requirements and Evaluation Criteria

To guarantee the feasibility and reliability of the monitoring model, the system must satisfy a rigorous set of technical standards spanning from the physical layer to the application layer.

Initially, concerning Data Integrity requirements, all telemetry data and door events must be tagged with accurate timestamps directly at the ESP32 microcontroller prior to publication over the MQTT network. The sensor placement must be logically justified and undergo a cross-calibration procedure against a standard reference thermometer to determine the device's margin of error.

Secondly, the system requires an advanced environmental compliance assessment mechanism Advanced Component. The server's REST API must not only support basic historical queries but also possess the capability to clearly distinguish between Warning levels and Sustained/Critical Excursions. The algorithm must empirically demonstrate the added value of utilizing a time-based exposure metric over an instantaneous warning threshold.

Furthermore, reliability and network Fault Tolerance are mandatory criteria. In real-world environments, Wi-Fi connectivity may be interrupted at any given

moment; therefore, the edge device is not permitted to lose audit data. The system must automatically initiate a ring buffer Local Buffering to store data offline and execute a synchronization process backfill to the database immediately upon network restoration, without disrupting the time axis of the charts.

Finally, the system will undergo a comprehensive evaluation through quantitative experimental scenarios. Mandatory evaluation criteria include: measuring sensor error, alarm delay, the precision of the exposure calculation, data-loss/recovery performance following a simulated network outage, and the incidence rate of false alarms under normal operational conditions.

Table 1.1: Summary of System Requirements and Evaluation Criteria

Module	**Core Technical Requirements**	**Evaluation Criteria / Evidence**
Hardware & Edge Device	Signal acquisition, pre-processing, accurate timestamping.	Experimental error measurement, accurate recording of door open events.
Protocol & Data	MQTT transmission, Time-series database storage.	Standardized JSON payload structure, seamless REST API historical retrieval.
Smart Monitoring	Thermal exposure assessment, incident severity classification.	Alert classification table, false alarm elimination algorithm.
Reliability	Local offline data buffering, synchronization upon network recovery.	Zero data loss following a simulated 30-second network disconnection.

CHAPTER 2: THEORETICAL BASIS AND HARDWARE SELECTION

2.1 Theoretical Basis of Cold Chain Monitoring

2.1.1 Impact of Temperature and Humidity on Sensitive Commodities

The cold supply chain is not merely about equipping cooling devices, but maintaining a stable micro-environment to protect the thermodynamic structure of sensitive products. For medical biologics, vaccines, and specialized pharmaceuticals, the standard temperature range is strictly regulated by the World Health Organization WHO typically from 2.0°C to 8.0°C. Maintaining this temperature range helps inhibit chemical decomposition reactions, prevent biological denaturation, and control the growth of microorganisms. When Hyperthermia occurs, i.e., the ambient temperature exceeds 8.0°C, the kinetic energy of molecules increases, leading to the breaking of weak bonds in the protein's peptide chains. Consequently, antibodies in vaccines lose their three-dimensional spatial structure, reducing or completely losing their immunogenic efficacy, and potentially transforming the pharmaceutical structure into impurities harmful to the body.

Conversely, freezing when the temperature drops below 2.0°C poses an even more severe physical risk. The formation of micro ice crystals permanently destroys the emulsion structure of vaccines containing aluminum adjuvants. When frozen, these adjuvant particles flocculate and cannot be reconstituted to their original state, causing clinical risks such as sterile abscesses at the injection site, respiratory failure complications, or severe anaphylactic shock for patients.

Besides temperature, relative humidity $RH\%$ is a secondary dynamic parameter but plays a key role in maintaining the integrity of the packaging system and storage microenvironment. Excess humidity environments greater than 75%RH promote water vapor condensation on warehouse surfaces, softening protective carton packaging, peeling identification labels, and creating ideal conditions for

mold spores to multiply, posing a risk of cross-contamination. Conversely, overly dry environmental conditions humidity below 30%RH increase surface vapor pressure, accelerating the diffusion and evaporation of solvents inside liquid pharmaceutical formulations or incompletely sealed medical specimens. Therefore, simultaneously monitoring both parameters is a prerequisite for a comprehensive assessment of cold chain quality.

To accurately quantify the actual damage to the product when environmental incidents occur, modern cold chain management theory has shifted from static threshold alarm models to dynamic assessment models, embodied by the Thermal Exposure metric. The basis of this method stems from the thermal inertia of goods; accordingly, the core temperature of the product does not change instantaneously with the variation of the surrounding air temperature. Therefore, the cumulative damage to the biochemical structure of the product is proportional to a product comprising the temperature deviation amplitude T and the duration of exposure t.

Mathematically, in continuous terms, the total cumulative exposure E during an observation period from time t_0 to t_1 is strictly represented through a definite integral formula:

$$E = \int_{t_0}^{t_1} f(T(t))dt$$

In this equation, the amplitude violation weighting function $f(T(t))$ relative to the safe temperature range $[T_{\min}, T_{\max}]$ acts as a mathematical filter, extracting only temperature values detrimental to product quality. Specifically, this piecewise function is defined as follows:

$$f(T(t)) = \begin{cases} T(t) - T_{MAX}, & \text{if } T(t) > T_{MAX} \\ T_{MIN} - T(t), & \text{if } T(t) < T_{MIN} \\ 0, & \text{if } T_{MIN} \le T(t) \le T_{MAX} \end{cases}$$

However, for IoT embedded monitoring systems executing on resource-constrained microcontroller platforms like the ESP32, environmental signals are collected as discrete data based on a fixed sampling period t. Consequently, to optimize computational capacity at the edge device Edge Computing, the continuous integral formula must be approximated into a discrete sum Degree-Time equation:

$$E = \sum_{i=1}^{N} f(T_i)t_i (unit: °C.minute)$$

Transitioning to this discrete calculation model allows the microcontroller to continuously accumulate the temperature error value after each sampling cycle ti without storing the entire time function. This mathematical theoretical basis is the core foundation for building the Thermal Finite State Machine Thermal FSM in the project's firmware source code. This algorithm helps the system automatically eliminate harmless short-term thermal fluctuations and only trigger alert escalations when the actual exposure level E exceeds the permissible tolerance limit of the stored goods.

2.2 Hardware Component Selection and Analysis

2.2.1 Central Microcontroller ESP32 DevKit V1 Integrated ESP32-WROOM-32 module



Figure 2.1: ESP32 DevKit V1 microcontroller board

The central microcontroller chosen to assume the role of the Edge Computing Node in the system is the ESP32 DevKit V1 development board. The heart of this board is the ESP32-WROOM-32 module, integrating a 32-bit dual-core Tensilica LX6 architecture processor operating at a clock frequency of 240 MHz. Unlike traditional 8-bit microcontrollers, the ESP32 is equipped with up to 520 KB of internal RAM and modern wireless connection standards including Wi-Fi 802.11 b/g/n and Bluetooth v4.2. This abundant hardware resource is a prerequisite allowing the device to execute multiple complex tasks in parallel. Specifically, the device can simultaneously execute signal noise filtering algorithms, process the state machine Thermal FSM calculating thermal exposure, and maintain a Ring

Buffer containing 100 offline data records to prepare for network disconnection incidents.

2.2.2 DHT22 Air Temperature and Humidity Measurement Sensor



Figure 2.2: DHT22 digital temperature and humidity sensor

To comprehensively monitor the overall microclimate environment inside the warehouse, the system uses the DHT22 digital sensor. This is a sensor line optimized to simultaneously measure two essential parameters: air temperature in the range from -40°C to 80°C with a rated error of 0.5°C, and relative humidity from 0% to 100%RH with an error of ±2%*RH* . The device communicates with the ESP32 microcontroller via a Single-Bus communication protocol connected at pin GPIO 27. Data collected from the DHT22 provides a complete picture of the ambient temperature surrounding the product, serving as input data for cold chain compliance assessment algorithms.

2.2.3 DS18B20 Accurate Core Temperature Sensor 1-Wire Protocol

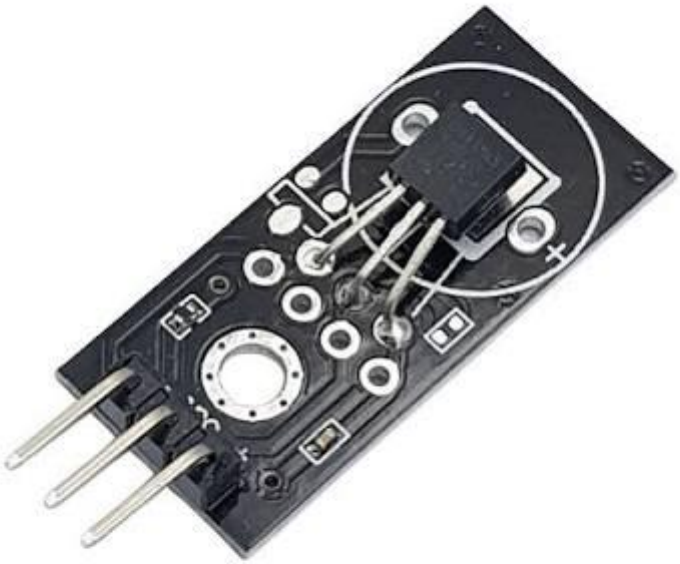


Figure 2.3: DS18B20 temperature probe

Besides the air environment sensor, the system integrates a DS18B20 sensor in the form of a waterproof metal probe to accurately measure deep temperature or temperature at the bottom area of the warehouse. The technical characteristics of the DS18B20 allow stable operation in an extremely wide measurement range from -55°C to 125°C with an accuracy of 0.5°C. The device uses the Dallas 1-Wire communication protocol connected directly to pin GPIO 14 of the ESP32. Notably, in terms of circuit design, because the 1-Wire protocol uses an Open-Drain transmission line structure, the voltage signal on the data transmission pin DQ is very susceptible to falling into a Floating Signal state if not clamped. To completely overcome this phenomenon, the research group applied a hardware design technique adding a 4.7k Pull-up Resistor, clamped between the DQ pin and the 3.3V positive power line. This technique helps keep the signal pulse amplitude always at a standard logic level, completely eliminating hardware noise errors that cause the sensor to return spurious values such as 85°C or 0°C.

2.2.4 28BYJ-48 Stepper Motor and ULN2003 IC



Figure 2.4: Simulation mechanism cluster including 28BYJ-48 Stepper Motor, ULN2003 Buffer Circuit, and Slide Switch

To simulate environmental disturbance scenarios due to warehouse operation procedures, the system designs a complete simulated warehouse door mechanism. This mechanism receives an activation signal from a Slide Switch acting as a door limit sensor, connected to pin GPIO 4. The main actuator is a 28BYJ-48 stepper motor with a basic step angle of 5.625°, combined with a reduction gearbox ratio of 1:64, creating a resolution of 2048 steps for a complete revolution 360°. To accurately simulate a 90° right-angle door opening stroke, the required number of steps is calculated according to the formula:

$$STEP_OPEN = 2048 \times \frac{90}{360} = 512 steps$$

The control signals from pins GPIO 19, 18, 5, and 21 of the ESP32 3.3V voltage level cannot directly pull the motor load, therefore the system uses the ULN2003 IC as a power buffer amplifier circuit. The ULN2003 microcircuit contains 7 Darlington Transistor pairs with a load current capacity of up to 500mA per

channel, serving to relay the logic signal into a sufficiently large current 5V to excite the phase coils of the stepper motor.

2.2.5 Isolated Cooling Control Mechanism 5V Relay Module



Figure 2.5: Optically isolated Relay Module and cooling load simulation mechanism

To react to environmental overheating incidents, the system uses a 1-channel Relay Module as a power switching mechanism for the emergency cooling system. This module integrates an Optocoupler isolation circuit, completely separating the control voltage section 3.3V logic from pin GPIO 26 of the ESP32 and the voltage section supplied to the consumable load 5V DC. During operation, when the ambient temperature exceeds the safe limit of 8.0°C, the ESP32 will output a signal to trigger the Normally Open NO contact of the Relay to close the circuit, thereby automatically supplying power to a cooling fan or a warning LED indicator.

2.3 Software Technology Basis

2.3.1 MQTT Communication Protocol and EMQX Broker

The MQTT communication protocol was selected as the core data exchange platform due to its ultra-lightweight nature and optimization for IoT systems operating in low-bandwidth environments. The system operates based on a distributed Publisher/Subscriber architecture, completely separating data acquisition devices from processing devices. In this model, the ESP32 microcontroller acts as the Publisher, responsible for collecting, packaging data into standard JSON format, and publishing it to designated Topics. To relay the information flow, the project uses the public EMQX server broker.emqx.io via TCP port 1883 as the routing Broker. At the end of the transmission flow, the Node.js

server and Web UI interface act as Subscribers, subscribing to the Topic coldchain/nhom07/telemetry to catch and process telemetry data as soon as it is published.

2.3.2 Node.js Execution Environment and SQLite Database

The central processing tier Backend is built on the Node.js environment combined with the Express framework. Characterized as an asynchronous JavaScript runtime environment based on Non-blocking I/O mechanism, Node.js provides the capability to process high-speed MQTT data streams with ultra-low latency. Accompanying the processing system is the SQLite embedded relational database management system, configured to store all data as a local file coldchain.db. The technical highlight in the database design stage is the establishment of a Composite Primary Key including two fields timestamp and device_id for the telemetry_data storage table. This constraint mechanism aims to completely prevent data duplication when the ESP32 edge device executes the buffer flush process to retroactively synchronize old data after a network disconnection incident.

2.3.3 WebSockets Technology and Chart.js Visualization

At the Frontend application layer, the system uses the MQTT.js library to establish real-time WebSockets connection streams via port 8083 of the EMQX Broker server. This protocol allows the browser to continuously receive pushed data from the server without performing page reloads, ensuring the continuity of the monitoring process. Concurrently, telemetry data is vividly visualized through the Chart.js library. The monitoring chart is complexly configured with two independent Y-axes left axis representing Temperature and right axis representing Relative Humidity, combined with an automatic data scrolling mechanism limited to a maximum of 20 display points. This dynamic display solution not only keeps the operating interface smooth but also prevents memory leaks on the browser when the system must operate continuously for many hours.

CHAPTER 3: SYSTEM ARCHITECTURE DESIGN

3.1 Overall System Architecture

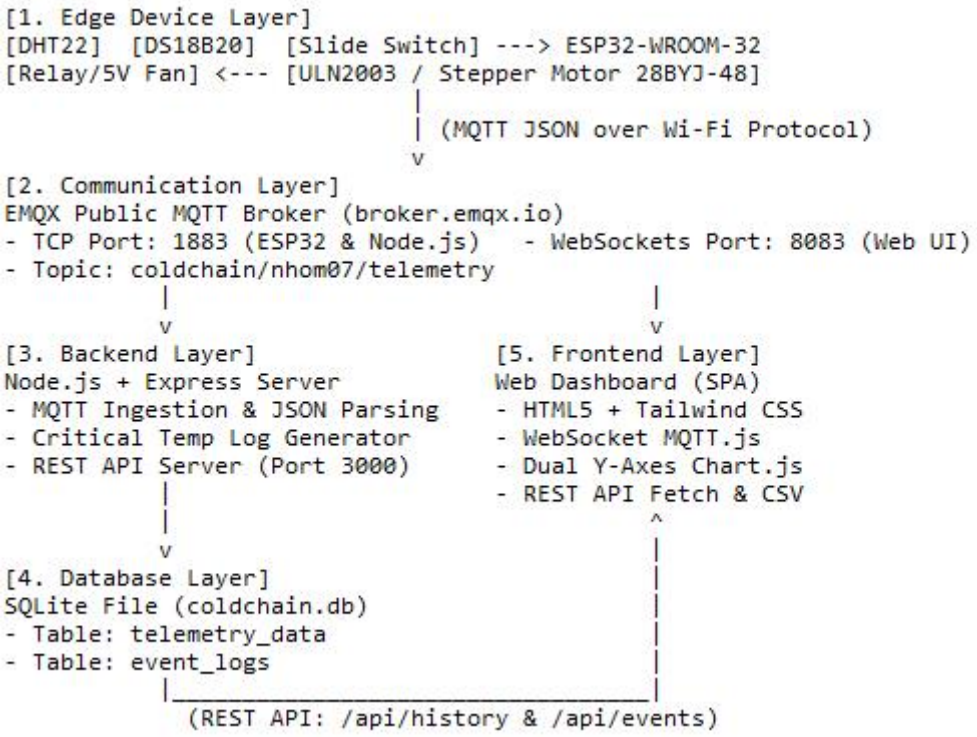


Figure 3.1: 5-Layer IoT Architecture diagram of the cold chain monitoring system

The cold supply chain monitoring system model is designed according to the standard 5-layer IoT architecture, ensuring high modularity and effective fault isolation. The foundational tier is the Edge Device Layer, where the ESP32 microcontroller directly interfaces with the sensor system DHT22, DS18B20 and actuators stepper motor, Relay to acquire physical parameters and respond to the local environment. Processed data is forwarded to the Communication Layer. Here, the EMQX Public Broker platform acts as a relay for JSON packets via the MQTT protocol with two parallel communication ports: TCP Port 1883 for data streams from physical devices and WebSockets Port 8083 for the user interface. The heart of the logical analysis process resides in the Backend Layer, where the Node.js and Express environments undertake packet dissection, evaluate temperature violations, and provide REST API endpoints Port 3000. Analyzed data is routed down to the Database Layer for persistent storage into a local SQLite file, clearly segregating time-series telemetry data and incident logs. Finally, the Frontend Layer manifests as a Single-Page Application SPA Web Dashboard. This layer visualizes all

information in real-time and permits administrators to interact and extract reports from all underlying tiers.

3.2 Data Flow Diagram DFD

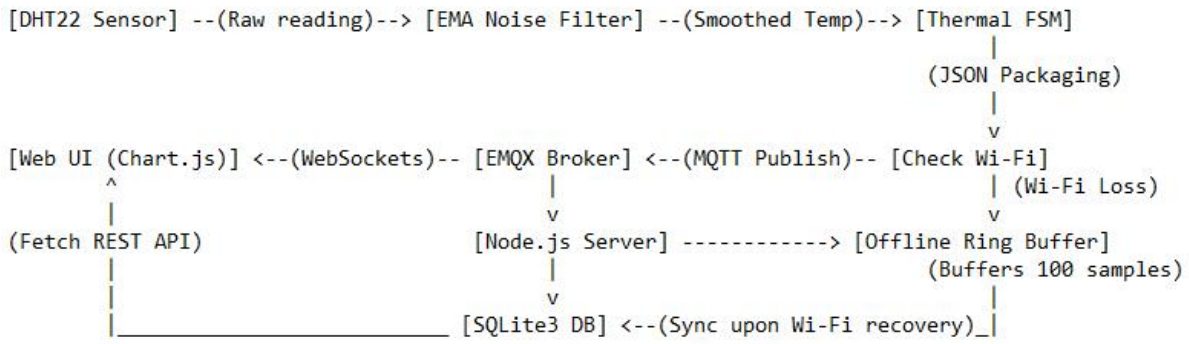


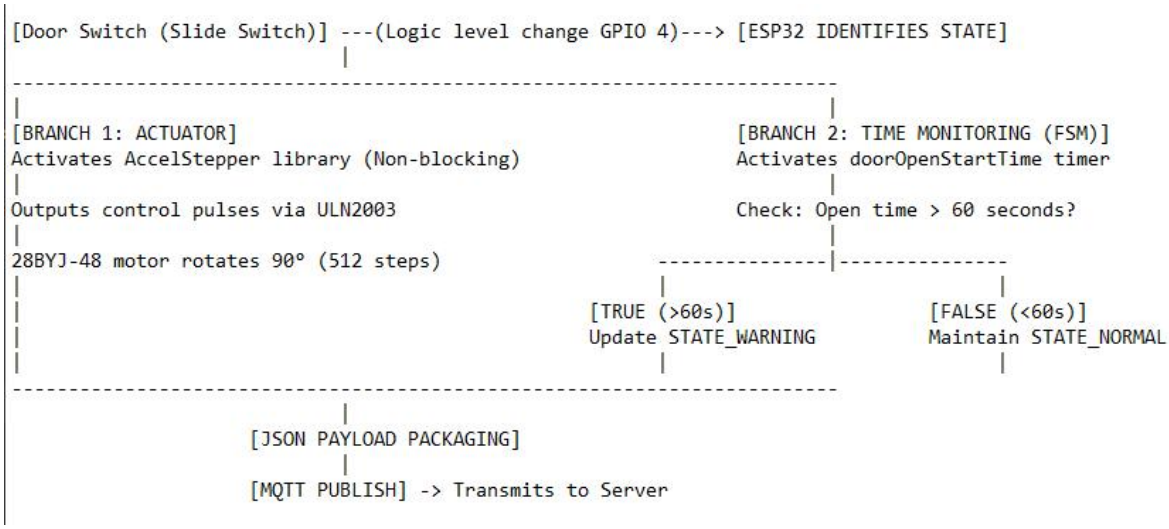
Figure 3.2: Life cycle and flow diagram of a telemetry data record

The life cycle of a telemetry data record commences with the acquisition of raw physical parameters at the sensor layer. This signal first passes through an Exponential Moving Average EMA noise filter at the microcontroller to smooth the data, eliminate transient noise spikes, and produce a refined temperature value. This value is subsequently fed into the state machine Thermal FSM to assess exposure levels before complete packaging into a valid JSON message. At the communication branch point, the data flow hinges entirely on the Wi-Fi connection state. If the communication network is stable, the JSON message is Published directly to the EMQX Broker, from which it is simultaneously distributed to the WebSockets stream instantly updating the Chart.js graph and the Node.js Server stream for recording into the SQLite database. Conversely, if the system detects a network disconnection, the data stream is diverted and temporarily stored into an Offline Ring Buffer directly on the ESP32's memory. Environmental acquisition continues cyclically without interruption. As soon as the connection is re-established, the system's main loop detects this and shifts to a Backfill Sync state. The algorithm continuously calls a buffer_pop function to flush all pending historical data and publish it to the Broker. Crucially, at the Backend server endpoint, to thoroughly prevent data overwriting or duplication due to transmission latency, the SQLite database is configured with a Composite Primary Key. This mechanism automatically verifies the timestamp and device_id value pair, ensuring each recovered record is perfectly stored at its correct location on the historical timeline.

3.3 Alert and Exception Handling Flow

3.3.1 Warehouse Door Open/Close Event Flow Door Event Flow

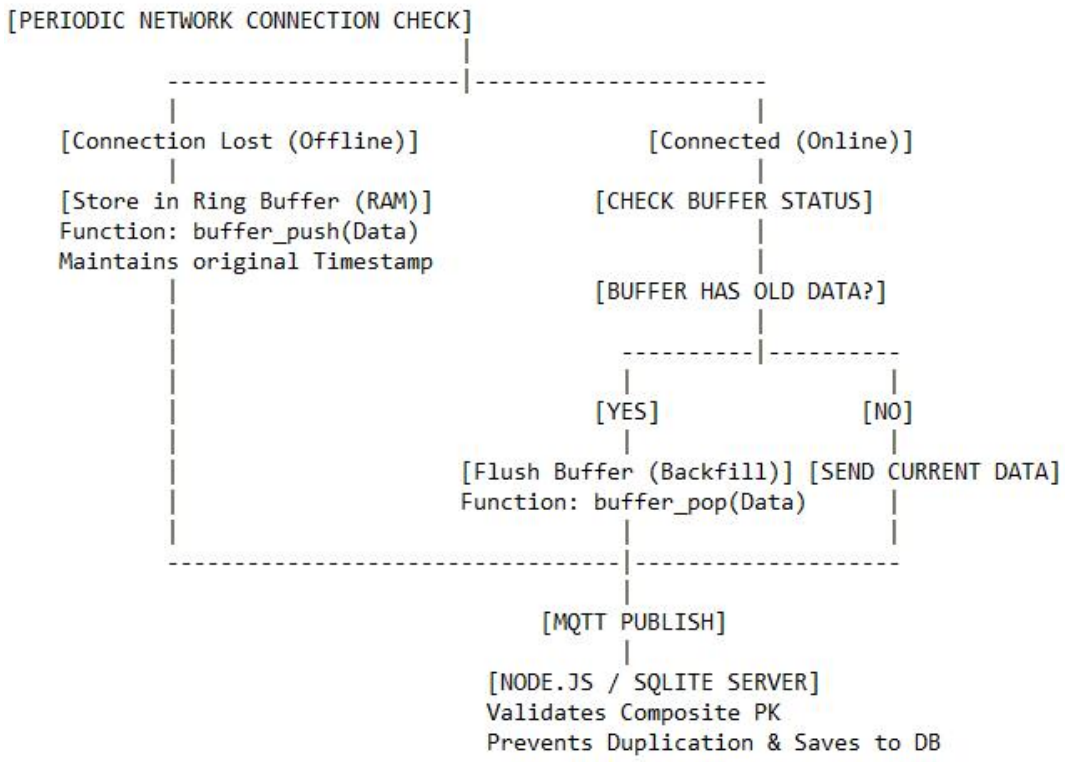
The door event handling flow is not merely recording a physical state, but also a synchronized coordination between a mechanical actuator and a time-monitoring state machine Thermal FSM. Initiating the cycle, when an operator interacts with the Slide Switch, the logic level change on pin GPIO 4 is immediately recognized by the microcontroller. Right at this moment, the execution thread branches into two parallel tasks to optimize response time. The first branch is responsible for controlling the actuator; the system calls a function from the AccelStepper library to output power pulses to the ULN2003 IC, controlling the 28BYJ-48 motor to rotate exactly 512 steps equivalent to a 90° opening angle in a non-blocking manner. In the second branch, the FSM state machine activates a real-time timer doorOpenStartTime. If the door remains open beyond a 60-second threshold 60000 ms, the system automatically escalates the status to a Warning level STATE_WARNING to signify the risk of heat loss. Finally, all information regarding physical door status and alert status is packed into a JSON string and published via the MQTT network to notify the central server.



3.3.2 Network Disconnection and Compensatory Synchronization Flow Network Failure & Sync Recovery

The offline data synchronization flow is a pivotal defense mechanism, assuring data chain continuity Data Integrity in inherently unstable IoT communication environments. Under standard operating conditions, the microcontroller continuously checks the Wi-Fi network signal and connection status to the MQTT Broker. When a communication disruption occurs, the system instantly activates

the Offline Buffering mechanism. Instead of discarding critical telemetry parameters, the algorithm invokes a buffer_push function to push the newest measurement data_including a standard UNIX timestamp precisely at the time of measurement_into a Ring Buffer with a capacity of 100 records on the ESP32's internal RAM. The environmental acquisition process continues cyclically without interruption. As soon as Wi-Fi connectivity is restored, the system's main loop recognizes this and transitions to a Backfill Sync state. The algorithm continuously calls a buffer_pop function to discharge all accumulated historical data and publish it to the Broker. Specifically, at the Backend server endpoint, to thoroughly prevent overwriting or duplicating data due to transmission latency, the SQLite database is established with a Composite Primary Key. This mechanism automatically verifies the timestamp and device_id value pair, ensuring each recovered record is flawlessly saved in its exact position on the historical timeline.



CHAPTER 4: HARDWARE AND SOFTWARE DESIGN AT THE BORDER – ESP32

4.1 Hardware Design and Wiring Diagram Hardware Architecture & Wiring Diagram

4.1.1 Overall Edge Node Hardware Architecture

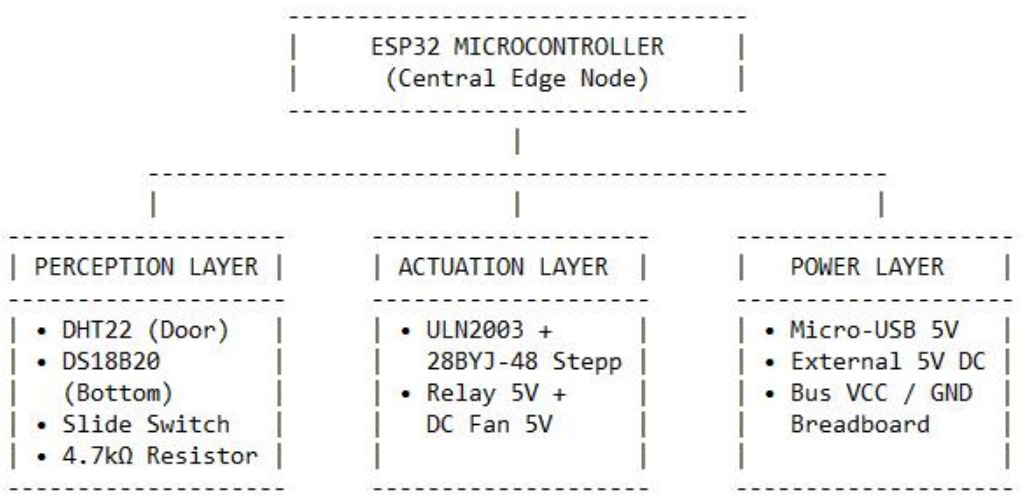


Figure 4.1: Central ESP32-WROOM-32 microcontroller and edge peripheral connection structure

The Edge Hardware Layer serves as the physical infrastructure for signal acquisition and direct task execution throughout the entire cold supply chain monitoring system. The core responsibilities of the edge device layer include: continuously collecting ambient air environmental parameters, directly measuring product temperature, monitoring the mechanical travel status of the warehouse door opening or closing, executing signal noise filtering algorithms, autonomously controlling emergency actuation mechanisms, and simultaneously packaging and transmitting data payloads via the wireless network.

The central microcontroller selected for this model is the ESP32-WROOM-32 board, which integrates a Tensilica LX6 dual-core processor with a clock frequency of 240 MHz, 520 KB of internal RAM, and modern wireless connectivity standards including Wi-Fi 802.11 b/g/n and Bluetooth v4.2. Owing to its abundant computational resources, the ESP32 is not merely a signal converter but functions as an Edge Computing Node. It independently operates data

smoothing algorithms, processes thermal exposure states, and manages local data buffering during network connection disruptions.

4.1.2 GPIO Pin Allocation and Planning for ESP32 Microcontroller

To ensure that transmitting and receiving signals do not conflict, the Input/Output GPIO pins on the ESP32 are strictly planned and allocated according to the electrical characteristics of each component:

Table 4.1: GPIO signal pin allocation plan on the ESP32

Module / Component	Component Pin	ESP32 Connection Pin	Operating Voltage Level
DHT22 Sensor	VCC / GND / DATA	3V3 / GND / GPIO 27	3.3V
DS18B20 Sensor	VCC / GND / DQ	3V3 / GND / GPIO 14	3.3V
Warehouse Door Switch	Signal / GND	GPIO 4 / GND	3.3V
28BYJ-48 Motor	IN1, IN2, IN3, IN4	GPIO 19, 18, 5, 21	5V VIN
Relay Module	IN / VCC / GND	GPIO 26 / VIN / GND	5V VIN
Dual Power Bus	3.3V Axis / 5V Axis	3V3 / VIN / GND	3.3V & 5V

4.1.3 Detailed Technical Wiring Analysis for Each Hardware Module

The physical wiring process is executed based on the principles of impedance matching and microcontroller protection. Initially, for the environmental sensor block, the data transmission pin DATA of the DHT22 is connected directly to GPIO 27. The DS18B20 metal probe sensor utilizes the DQ pin connected to GPIO 14. Specifically at this communication node, the system is designed with an additional 4.7k Pull-up Resistor bridging the DQ pin to the 3.3V positive power rail. This solution aims to meet the requirements of the 1-Wire protocol, ensuring the signal does not experience logic level floating. Furthermore, the simulated warehouse door switch Slide Switch block has its signal pin connected to GPIO 4 and the remaining pin connected to common ground GND. Thanks to the internal pull-up resistor configuration INPUT_PULLUP within the source code, the switch

state is clearly defined: LOW 0 when the door is closed and HIGH 1 when the door is open.

Regarding the electromechanical system, the ULN2003 buffer circuit controlling the 28BYJ-48 stepper motor receives pulses from the ESP32 via pins IN1 to IN4, corresponding to GPIO 19, 18, 5, and 21. The motor's 5-wire plug is connected directly to the standard slot on the ULN2003 microcircuit, while the power supply is drawn directly from the 5V axis VIN to avoid overloading the ESP32's current capacity. Similarly, the Relay power circuit controlling the cooling fan receives the activation signal from GPIO 26, switching the 5V current supplied to the cooling mechanism during an incident. A particularly crucial technical highlight in the hardware design is the principle of establishing a Dual Power Bus and suppressing Flyback Noise. The stepper motor and Relay inductor require a large starting current 300-600mA during operation. To protect the microcontroller, the research team established two completely separate power bus rails on the Breadboard: a 3.3V axis supplying clean reference current for the sensors, and a dedicated 5V VIN axis for the power block. This separation helps eliminate temporary voltage drops Brownout Reset and protects the ESP32's GPIO pins from induced reverse currents.

4.1.4 Sensor Calibration Procedure with Reference Equipment

Prior to entering official trial operation, the DHT22 and DS18B20 temperature sensor systems must undergo a rigorous cross-calibration procedure with a Reference Thermometer placed in the same microclimate testing environment. At each sampling point, the sensor's deviation is recorded. The absolute error at the i-th measurement point is determined by the mathematical formula:

$$Error_i = | T_{sensor,i} - T_{reference,i} |$$

Based on this dataset, the Mean Absolute Error MAE index across a total of N=30 test samples is calculated using the formula:

$$MAE = \frac{1}{N} \sum_{i=1}^{N} | T_{sensor,i} - T_{reference,i} |$$

The experimental calibration results indicate that the MAE coefficient achieved is below 0.25°C, demonstrating the high reliability and stability of the sensor system in measuring temperature levels within the safe range of 2.0°C to 8.0°C.

4.2 ESP32 Firmware Software Algorithm Source Code Analysis

4.2.1 Exponential Moving Average EMA Noise Filtering Algorithm

To resolve the issue of transient spike noise arising from the physical signal reading process of the DHT22 sensor, the source code integrates a discrete EMA digital filtering algorithm. The mathematical equation of the filter is represented by a recurrence relation where is the smoothing coefficient:

$$smoothedTemp_k = \alpha \cdot T_{cal} + (1-\alpha) \cdot smoothedTemp_{k-1}$$

With a smoothing coefficient = 0.2, the algorithm retains 80% of the weight from the previous temperature history, thereby helping to smooth the signal while ensuring it closely tracks reality. The C++ source code structure is implemented as follows:

```
float smoothedTemp = -999.0;

float readTemperature(void) {
  float rawTemp = dht.readTemperature();

  if (isnan(rawTemp)) {
    Serial.println("[CANH BAO] Loi doc cam bien DHT22!");
    return (smoothedTemp == -999.0) ? 0.0 : smoothedTemp;
  }

  float offset_temp = 0.0;
  float calTemp = rawTemp + offset_temp;

  if (smoothedTemp == -999.0) {
    smoothedTemp = calTemp;
  } else {
    smoothedTemp = (calTemp * 0.2) + (smoothedTemp * 0.8);
  }
  return smoothedTemp;
}
```

Figure 4.2: Code snippet reference for EMA filter algorithm

4.2.2 Thermal Finite State Machine Thermal FSM & Thermal Exposure Algorithm

The focal point of the edge control logic is the updateThermalFSM function. This function continuously monitors the temperature deviation against the safe range

2.0°C - 8.0°C and multiplies it by the time differential to calculate the accumulated total exposure totalExposure. Concurrently, it monitors a real-time timer to activate a warning state if the warehouse door remains open continuously beyond 60 seconds DOOR_ALERT_MS. The FSM state machine automatically transitions back and forth between three states: STATE_NORMAL, STATE_WARNING, and STATE_VIOLATION.

```
const float T_MAX = 8.0;
const float T_MIN = 2.0;
const float EXPOSURE_LIMIT = 30.0;
const unsigned long DOOR_ALERT_MS = 60000;

ThermalState currentState = STATE_NORMAL;
float totalExposure = 0.0;
unsigned long lastCalcTime = 0;
unsigned long doorOpenStartTime = 0;

void updateThermalFSM(float currentTemp, bool isDoorOpen) {
  unsigned long currentTime = millis();

  if (lastCalcTime == 0) { lastCalcTime = currentTime; return; }

  float dt_minutes = (currentTime - lastCalcTime) / 60000.0;
  if (currentTemp > T_MAX) {
    totalExposure += (currentTemp - T_MAX) * dt_minutes;
  } else if (currentTemp < T_MIN) {
    totalExposure += (T_MIN - currentTemp) * dt_minutes;
  }
  lastCalcTime = currentTime;

  if (isDoorOpen && doorOpenStartTime == 0) {
    doorOpenStartTime = currentTime;
  } else if (!isDoorOpen) {
    doorOpenStartTime = 0;
  }
  bool doorWarning = (isDoorOpen && (currentTime - doorOpenStartTime > DOOR_ALERT_MS));

  switch (currentState) {
    case STATE_NORMAL:
      if (totalExposure >= EXPOSURE_LIMIT) currentState = STATE_VIOLATION;
      else if (currentTemp > T_MAX || currentTemp < T_MIN || doorWarning) currentState = STATE_WARNING;
      break;
    case STATE_WARNING:
      if (totalExposure >= EXPOSURE_LIMIT) currentState = STATE_VIOLATION;
      else if (currentTemp <= T_MAX && currentTemp >= T_MIN && !doorWarning) currentState = STATE_NORMAL;
      break;
    case STATE_VIOLATION:
      break;
  }
}
```

Figure 4.3: Code snippet reference for Thermal FSM

4.2.3 Non-blocking Stepper Motor Control

To ensure the ESP32 can continuously read sensors and maintain Wi-Fi connectivity while the door mechanism is in motion, the team applied a non-blocking control method via the AccelStepper library. When the slide switch changes state, the target variable is set to 512 steps equivalent to 90° and the stepper.run function is continuously called within the main loop. When the motor

completes its travel, the stepper.disableOutputs command is called to cut off the current flowing through the coils, helping to prevent heat generation for the Driver and save energy.

```
// Hàm giám sát công tắc gạt và kích hoạt góc quay mô-tơ
void handleDoorControl(void)
{
  bool switchState = (digitalRead(DOOR_PIN) == LOW);

  if (switchState && !doorState) {
    doorState = true;
    stepper.enableOutputs();
    stepper.moveTo(STEPS_OPEN);
    Serial.println("[CUA] GAT CONG TAC -> MO CUA (Quay tien 90 do)");
  }
  else if (!switchState && doorState) {
    doorState = false;
    stepper.enableOutputs();
    stepper.moveTo(STEPS_CLOSED);
    Serial.println("[CUA] GAT NGUOC LAI -> DONG CUA (Quay lui ve 0 do)");
  }

  // Nếu chưa tới đích thì tiếp tục chạy bước tiếp theo
  if (stepper.distanceToGo() != 0) {
    stepper.run();
  } else {
    // Tới đích rồi thì ngắt điện nghỉ ngơi
    stepper.disableOutputs();
  }
}

unsigned long getUnixTime(void) {
  // Dùng millis() tạo timestamp tịnh tiến chống lỗi SQLITE_CONSTRAINT trùng lặp
  return (millis() / 1000) + 1773700000;
}
```

Figure 4.4: Code snippet reference for Stepper Control

4.2.4 Offline Ring Buffer Management

The network loss defense mechanism is actualized through a ring array data structure named dataBuffer, capable of storing a maximum of 100 TelemetryRecord structure records. When the system detects an MQTT connection loss, the acquisition thread does not halt but immediately calls the buffer_push function to insert new data into the buffer. As soon as the network recovers, the main loop uses the buffer_pop function to sequentially flush the entire telemetry history along with the original past timestamps back to the EMQX Broker server.

```
// ======================================
#define BUFFER_SIZE 100

typedef struct {
  TelemetryRecord buffer[BUFFER_SIZE];
  int head;
  int tail;
  int count;
} OfflineBuffer;

OfflineBuffer dataBuffer = { .head = 0, .tail = 0, .count = 0 };

bool buffer_push(OfflineBuffer *cb, TelemetryRecord record) {
  if (cb->count >= BUFFER_SIZE) {
    cb->tail = (cb->tail + 1) % BUFFER_SIZE;
  } else {
    cb->count++;
  }
  cb->buffer[cb->head] = record;
  cb->head = (cb->head + 1) % BUFFER_SIZE;
  return true;
}

bool buffer_pop(OfflineBuffer *cb, TelemetryRecord *record) {
  if (cb->count == 0) return false;
  *record = cb->buffer[cb->tail];
  cb->tail = (cb->tail + 1) % BUFFER_SIZE;
  cb->count--;
  return true;
}

bool buffer_isEmpty(OfflineBuffer *cb) {
  return cb->count == 0;
}
```

Figure 4.5: Code snippet reference for Ring Buffer

4.2.5 Main Loop Execution Sequence

The loop function represents the perpetual life cycle loop of the system. It is responsible for calling functions to maintain the MQTT connection, updating the non-blocking stepper motor position, and reading environmental signals based on a 5000 ms sampling cycle SAMPLE_INTERVAL. Here, the Relay alert activation signal is also determined, the FSM is updated, and the final payload is routed to be Published or pushed into the ring buffer depending on the network system's availability.



```
void loop() {
  // Duy trì kết nối mạng
  if (WiFi.status() == WL_CONNECTED) {
    if (!mqttClient.connected()) reconnect_mqtt();
    mqttClient.loop();
  }

  // Quản lý cơ cấu chuyển động cửa bước (Non-blocking)
  handleDoorControl();

  // Chu kỳ lấy mẫu và bắn dữ liệu lên máy chủ
  unsigned long currentTime = millis();
  if (currentTime - lastSampleTime >= SAMPLE_INTERVAL) {
    lastSampleTime = currentTime;

    // 1. Đọc dữ liệu môi trường & trạng thái cửa
    float t = readTemperature();
    float h = readHumidity();
    bool door = doorState;

    // 2. Logic cảnh báo môi trường (Đèn/Quạt/Relay)
    if (t > 8.0 && !currentCoolingState) {
      digitalWrite(FAN_PIN, HIGH);
      currentCoolingState = true;
    } else if (t <= 8.0 && currentCoolingState) {
      digitalWrite(FAN_PIN, LOW);
      currentCoolingState = false;
    }

    // 3. Cập nhật FSM tích lũy độ phơi nhiễm
    updateThermalFSM(t, door);

    TelemetryRecord record;
    record.timestamp = getUnixTime();
    record.temperature = t;
    record.humidity = h;
    record.doorOpen = door;
    record.coolingState = currentCoolingState;
    record.accumulatedExposure = totalExposure;
    record.state = currentState;

    // 4. Gửi dữ liệu MQTT
    if (isMqttConnected()) {
      sendToMqtt(&record, false);
      while (!buffer_isEmpty(&dataBuffer)) {
        TelemetryRecord oldRecord;
        if (buffer_pop(&dataBuffer, &oldRecord)) {
          sendToMqtt(&oldRecord, true);
          delay(50);
        }
      }
    } else {
      buffer_push(&dataBuffer, record);
      Serial.printf("[OFFLINE] Luu vao bo dem. (Data ket: %d/%d)\n",
                    dataBuffer.count, BUFFER_SIZE);
    }
  }
}
```

Figure 4.6: Code snippet reference for Main Loop

CHAPTER 5: BACKEND, DATABASE, AND API COMMUNICATION

The Backend tier serves as the central processing component in the architecture of the cold-chain monitoring system. The core functions of this tier include data ingestion MQTT Ingestion from edge devices, data validation, telemetry processing, threshold violation detection, and data storage in the database. At the same time, the Backend provides Application Programming Interfaces REST APIs to support data retrieval from the monitoring application Dashboard. The system is deployed on a Node.js environment combined with the Express framework and utilizes an embedded SQLite database to optimize processing performance with low latency.

5.1 Database Design

To meet the requirement of systematically storing monitoring data and supporting efficient historical retrieval, the database is designed based on the SQLite relational database management system. The data model is separated into independent physical tables to optimize query performance for time-series data and isolate system events. The database schema includes:

Column Name	Data Type	Description
timestamp	INTEGER	Standard UNIX Epoch timestamp Seconds, serving as the time identifier key.
device_id	TEXT	Edge device identifier eg., ESP32.COLD_01.
temperature	REAL	Measured temperature value Unit: °C.
humidity	REAL	Relative humidity value Unit: %.
door_open	BOOLEAN	Door status: true Open, false Closed.
cooling_state	BOOLEAN	Cooling mechanism status: true Active, false Off.

Table 5.1: Structural schema of the telemetry_data table (Time-series storage)

Column Name	Data Type	Description
timestamp	INTEGER	Time point when the system records an incident Standard UNIX Epoch.
device_id	TEXT	Identifier of the device where the incident occurred.
event_type	TEXT	Incident classification e.g., CRITICAL_TEMP.
description	TEXT	Detailed description of the violation state e.g., Abnormal temperature: 8.3°C.

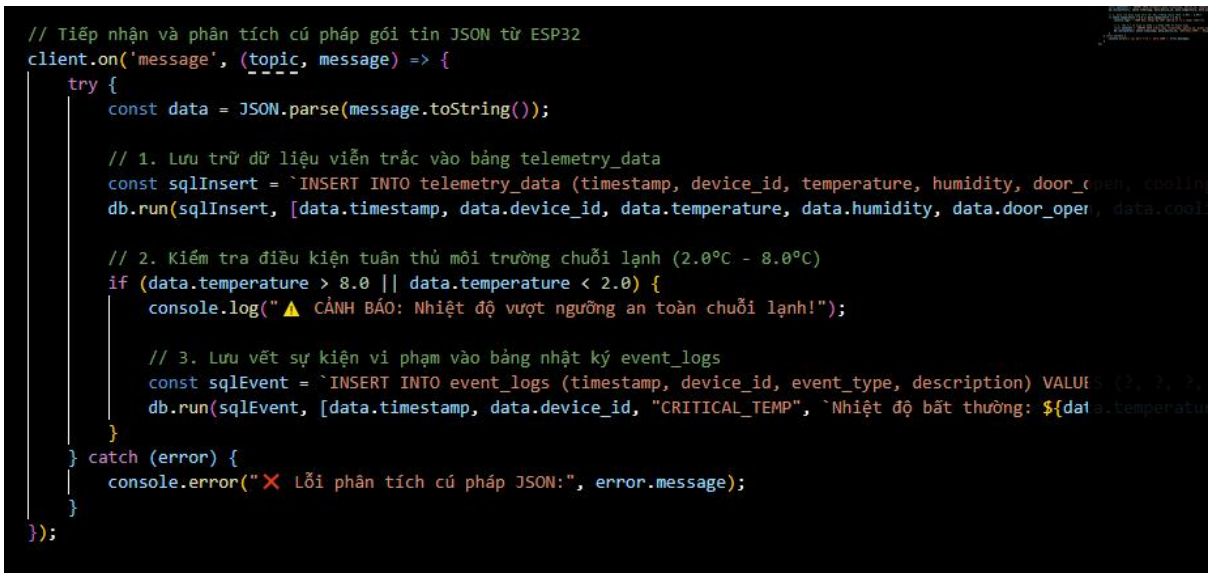
Table 5.2: Structural schema of the event_logs table Event log

Separating these data fields independently allows the system to minimize complexity in the storage structure while optimizing retrieval speed for data points when the client application requests chart rendering and historical violation queries.

5.2 MQTT Ingestion and Data Processing

The data ingestion process MQTT Ingestion is performed via the MQTT protocol, connecting directly to the EMQX Broker. Data is routed based on the standardized Topic convention coldchain/nhom07/telemetry, combined with the packet format JSON Payload from the edge device.

When the Broker distributes a new message, the server carries out JSON parsing and data normalization. The internal processing flow is not limited to format conversion for storage, but also integrates real-time environmental monitoring and status evaluation algorithms. Specifically, the system compares measured values with set thresholds; if the temperature falls outside the safe range below 2.0°C or above 8.0°C, the Backend automatically detects and classifies this as a CRITICAL_TEMP event.



```
// Tiếp nhận và phân tích cú pháp gói tin JSON từ ESP32
client.on('message', (topic, message) => {
    try {
        const data = JSON.parse(message.toString());

        // 1. Lưu trữ dữ liệu viễn trắc vào bảng telemetry_data
        const sqlInsert = `INSERT INTO telemetry_data (timestamp, device_id, temperature, humidity, door_
        db.run(sqlInsert, [data.timestamp, data.device_id, data.temperature, data.humidity, data.door_oper

        // 2. Kiểm tra điều kiện tuân thủ môi trường chuỗi lạnh (2.0°C - 8.0°C)
        if (data.temperature > 8.0 || data.temperature < 2.0) {
            console.log("⚠️ CẢNH BÁO: Nhiệt độ vượt ngưỡng an toàn chuỗi lạnh!");

            // 3. Lưu vết sự kiện vi phạm vào bảng nhật ký event_logs
            const sqlEvent = `INSERT INTO event_logs (timestamp, device_id, event_type, description) VALUE
            db.run(sqlEvent, [data.timestamp, data.device_id, "CRITICAL_TEMP", `Nhiệt độ bất thường: ${dat
        }
    } catch (error) {
        console.error("❌ Lỗi phân tích cú pháp JSON:", error.message);
    }
});
```

Figure 5.1: Source code snippet of JSON parsing logic and temperature violation tracking

When a violation condition is confirmed, the execution flow triggers a command to record the event into the event_logs table. This mechanism serves to trace incident history, ensuring transparency and supporting root-cause analysis for environmental anomalies.

5.3 REST API Communication and Response Structure

The REST API serves as the core software interface connecting the Backend and the Application tier Dashboard. This architecture decouples the data collection flow via MQTT from the information retrieval flow, thereby enhancing system security and scalability. The server provides two primary endpoints:

API /api/history: Executes database queries to extract the 50 most recent time records from the telemetry_data table. This method of limiting retrieved data volume helps maintain chart rendering performance on the user interface without causing transmission bandwidth congestion.

API /api/events: Provides a list of violation events extracted from the log table, supporting client applications in generating compliance status reports based on recorded incidents.

The APIs are designed adhering to a standardized Response Envelope format and integrated with a CORS Cross-Origin Resource Sharing mechanism to allow secure cross-platform interaction. The returned JSON data structure is formatted as follows:

```
JSON

{
  "success": true,
  "data": [
    {
      "timestamp": 1789489174,
      "device_id": "ESP32_COLD_01",
      "temperature": 8.3,
      "humidity": 60.0,
      "door_open": 0,
      "cooling_state": 1
    }
  ]
}
```


Figure 5.2: Standardized JSON Response Payload structure framework from REST API

Experimental results on the browser demonstrate that the Backend system has successfully routed and responded with data according to the specified structure, meeting the requirement of providing a reliable historical information stream for the display dashboard application.

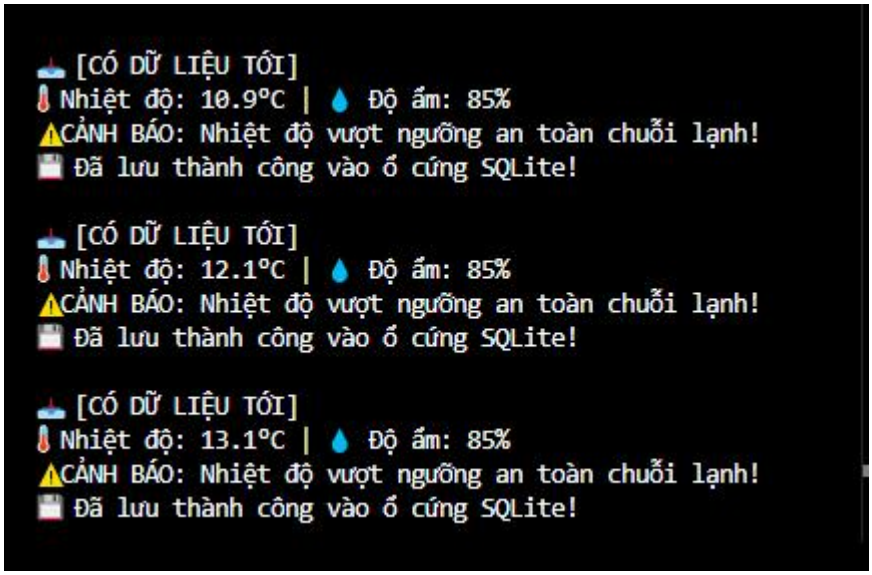


Figure 5.3: Successful result of API query exporting historical data /api/history

CHAPTER 6: DESIGN OF INTELLIGENT MONITORING DASHBOARD

The Application Layer Web Dashboard serves as the direct interaction interface between the cold storage operator and the entire underlying IoT system. The core functions of this layer include real-time telemetry data visualization, reflecting the operational status of peripheral devices, automatically triggering visual alerts upon environmental threshold violations, and providing tools for historical data exploration. The interface is built using a Single-Page Application SPA architecture based on pure HTML5 combined with the Tailwind CSS UI framework, the Chart.js visualization library, and the MQTT.js library communicating via WebSockets. This combination helps optimize page load speeds and ensures responsiveness across various display devices.

6.1 User Interface Design (UI/UX)

To ensure intuitiveness and assist operators in easily grasping the cold storage status in the shortest time, the Dashboard interface is designed according to a Responsive Grid System divided into 3 main functional areas: Header, Status Cards, and Time-Series Visualization Chart.

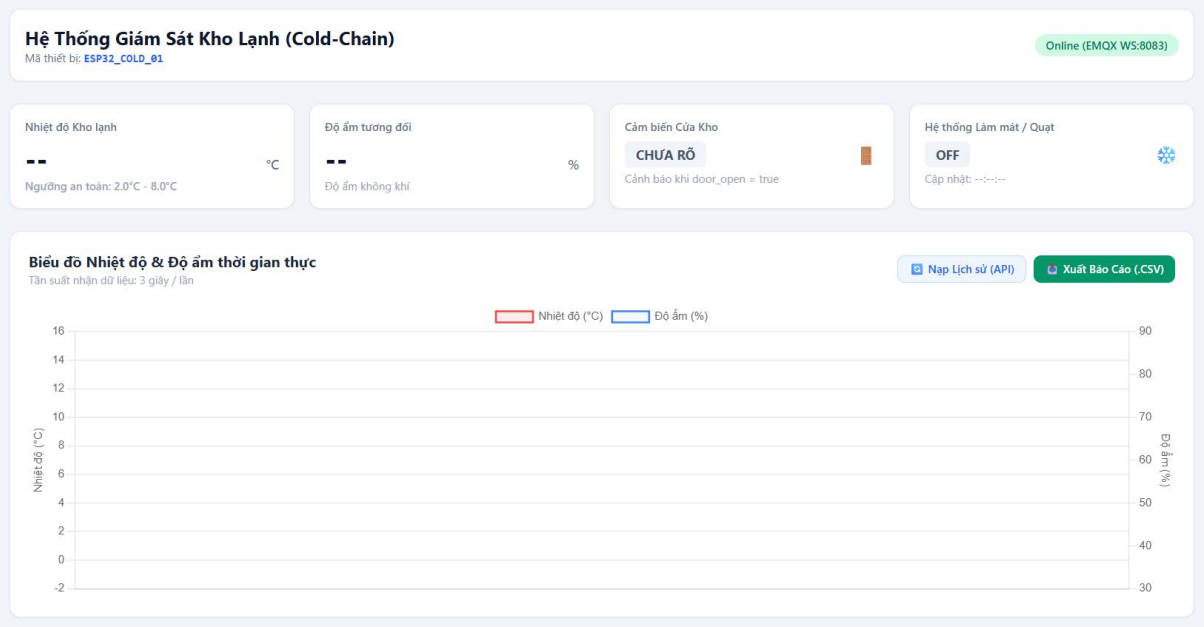


Figure 6.1: Overall layout diagram of the cold-chain monitoring Dashboard interface

The detailed structure of the interface components on the Dashboard is specified in Table 6.1:

UI Component	ID	Display Type	Description
Connection Badge	connection-badge	Status card	Displays MQTT connection status: Connecting... Gray, ONLINE Green, OFFLINE Red.
Temperature Card	temp-card, temp-val	Number & Color-changing card	Displays cold storage temperature °C. Turns red and flashes animate-pulse when exceeding the safe range of $2.0^{\circ}C - 8.0^{\circ}C$.
Humidity Card	hum-val	Quantitative number	Displays relative humidity %RH in the cold storage air.
Door Card	door-card, door-status	door-card, door-status	Status card & Icon | Description: Displays CLOSED Gray or OPEN Warning yellow when door_open = true combined with a icon.
Cooling Fan Card	cooling-status	Status card & Icon	Displays fan Relay status ON / OFF and the timestamp of the last packet update last-update.

Table 6.1: Detailed UI components breakdown

6.2 Real-time Connection via MQTT/WebSockets and Alert Logic

Since standard web browsers do not support pure TCP Socket protocols, the client application utilizes the MQTT over WebSockets mechanism via port 8083 of the EMQX Broker. This enables data transmission directly from the Broker to the browser with extremely low latency < 50ms.

Once the WebSocket connection is successfully established, the application subscribes to the Topic coldchain/nhom07/telemetry. The process of receiving and handling violation events is executed completely asynchronously on the client side.

```
// Trích đoạn logic tiếp nhận gói tin MQTT qua WebSockets và kích hoạt Cảnh báo
client.on('message', (topic, message) => {
  const payload = JSON.parse(message.toString());

  // 1. Cập nhật số liệu tức thời lên giao diện DOM
  document.getElementById('temp-val').innerText = payload.temperature.toFixed(1);
  document.getElementById('hum-val').innerText = payload.humidity.toFixed(1);

  // 2. Logic kiểm tra dải an toàn (2.0°C - 8.0°C) và kích hoạt hiệu ứng Cảnh báo thị giác
  const tempCard = document.getElementById('temp-card');
  if (payload.temperature > 8.0 || payload.temperature < 2.0) {
    // Vi phạm ngưỡng: Chuyển nền đỏ rực và nhấp nháy liên tục (Flash Alert)
    tempCard.className = "bg-rose-500 text-white p-4 rounded-xl shadow-lg animate-pulse";
    document.getElementById('temp-status').innerText = "⚠ VI PHẠM NHIỆT ĐỘ AN TOÀN!";
  } else {
    // Trạng thái đạt chuẩn Chuỗi lạnh
    tempCard.className = "bg-white p-4 rounded-xl shadow-sm border border-slate-200";
    document.getElementById('temp-status').innerText = "Ngưỡng an toàn: 2.0°C - 8.0°C";
  }
});
```

Figure 6.2: Source code snippet of JSON parsing logic and visual Alert state transition on the Dashboard

6.3 Real-time Data Visualization (Chart.js)

To assist managers in evaluating environmental fluctuation trends, the system integrates a real-time multi-axis line chart via the HTML5 <canvas id=“telemetryChart”> tag. The chart presents two parameters side-by-side: Temperature °C on the left vertical axis and Humidity %RH on the right vertical axis.

```
function updateChart(data) {
  const timeStr = data.timestamp
    ? new Date(data.timestamp * 1000).toLocaleTimeString()
    : new Date().toLocaleTimeString();

  telemetryChart.data.labels.push(timeStr);
  telemetryChart.data.datasets[0].data.push(data.temperature);
  telemetryChart.data.datasets[1].data.push(data.humidity);


  if (telemetryChart.data.labels.length > MAX_CHART_POINTS) {
    telemetryChart.data.labels.shift();
    telemetryChart.data.datasets[0].data.shift();
    telemetryChart.data.datasets[1].data.shift();
  }

  telemetryChart.update('none');
}
```

Figure 6.3: Utilizing the Sliding Window algorithm and Epoch Timestamp processing

6.4 Integration of Historical REST API and CSV Report Export Functionality

In parallel with the real-time data transmission stream via WebSockets, the Dashboard interface tightly integrates with the Backend tier Node.js & SQLite through two extended features:

1. **Historical Retrieval via REST API (fetchHistoryData)**

When the user triggers the "🔄 Load History API" button, the application executes an HTTP GET query to the endpoint http://localhost:3000/api/history. The received data is unwrapped from the Response Envelope {success: true, data: [...]} and re-renders the 50 most recent time points onto the chart, supporting display data recovery when the user reloads the webpage.

2. **Text Report Export (exportCSV)**

This function allows packaging the entire measurement time-series currently displayed on the chart into a .csv format file right on the client side, without consuming processing resources on the Backend server. The conversion process utilizes the Data URI Scheme technique:

$$\text{CSV Content} = \text{"Time, Temperature(C), Humidity(\%)} \backslash n\text{"} + \sum_{i=1}^{N} (\text{label}_i + \text{temp}_i + \text{hum}_i)$$

```
function exportCSV() {
  const labels = telemetryChart.data.labels;
  const temps = telemetryChart.data.datasets[0].data;
  const hums = telemetryChart.data.datasets[1].data;

  if (labels.length === 0) {
    alert('Chưa có dữ liệu nào trên biểu đồ để xuất!');
    return;
  }

  let csvContent = "data:text/csv;charset=utf-8,Thoi Gian,Nhiet Do (C),Do Am (%)\n";

  for (let i = 0; i < labels.length; i++) {
    csvContent += `${labels[i]},${temps[i]},${hums[i]}\n`;
  }

  const encodedUri = encodeURI(csvContent);
  const link = document.createElement("a");
  link.setAttribute("href", encodedUri);
  link.setAttribute("download", `coldchain_log_${Date.now()}.csv`);
  document.body.appendChild(link);
  link.click();
  document.body.removeChild(link);
}
```

Figure 6.4: Logic for generating files and triggering CSV report downloads from Frontend cached data

CHAPTER 7: EXPERIMENTAL RESULTS AND EVALUATION

The experimental process was conducted to verify and evaluate the operational capabilities of the entire prototype system, from the physical device layer to the user interface application layer. The tests were designed based on three operating scenarios closely matching real-world cold-chain practices, including: normal operating conditions, responding to temperature spike incidents, and self-recovery capabilities following communication network disruptions.

7.1 Scenario 1: Normal Operation and Storage Door Control

To verify system stability under standard operating conditions, the first scenario focused on evaluating environmental measurement capability and mechanical mechanism accuracy. Under laboratory conditions, temperature and humidity were maintained within safe limits temperature < 8.0°C. Results showed that the DHT22 sensor, after passing through an Exponential Moving Average EMA filter, provided

a stable data stream; the Dashboard interface accurately displayed temperature benchmarks every 5 seconds with a white status card Normal.

Next, a physical action on the Slide Switch was performed to simulate an employee opening the cold storage door. As soon as the switch toggled its logic state, the ESP32 microcontroller immediately issued a non-blocking control command to the ULN2003 driver circuit. Actual observation showed that the 28BYJ-48 stepper motor precisely executed a 512-step rotation, equivalent to a 90° opening angle, and then automatically cut off power to prevent heat generation. Simultaneously, the door status card on the Dashboard instantly changed to a warning yellow label “OPEN”, demonstrating near-zero transmission latency from the edge device to the Web UI via MQTT WebSockets.



Figure 7.1: Web interface displaying normal status and simulating a 90° door opening mechanism

7.2 Scenario 2: Temperature Threshold Violation and Cooling System Response

The second scenario was designed to evaluate the sensitivity of the Thermal Finite State Machine Thermal FSM and the chain reaction of the End-to-End system during environmental incidents. The experiment was conducted by manually applying heat directly to the DHT22 sensor surface to force the temperature above the 8.0°C limit. As soon as the temperature condition was breached, a series of protective mechanisms were activated simultaneously.

At the edge device layer, the FSM algorithm recorded a sharp increase in thermal exposure level, and the ESP32 automatically output a logic HIGH signal via GPIO pin 26 to activate the Relay Module, closing the circuit and supplying 5V power to

run the cooling fan system to rescue the incident. At the user interface layer, telemetry data carrying the violation message instantly caused the Dashboard to switch the temperature display block to a bright red background, combined with a continuous flashing effect animate-pulse to attract the operator's attention. Simultaneously at the server layer, the Node.js Backend, upon capturing the violation packet, automatically generated an SQL query and successfully inserted an event record with the code CRITICAL_TEMP into the event_logs table of the SQLite database, permanently logging the incident for post-audit purposes.



Figure 7.2: System activating a red visual alert on the Web and turning on the cooling fan via Relay

7.3 Scenario 3: Disconnection and Data Backfill Recovery Mechanism

The third scenario was a test to evaluate network fault tolerance and the system's time-series preservation mechanism. In this scenario, the Wi-Fi router providing connectivity to the ESP32 was intentionally disconnected for 30 seconds. The Dashboard interface immediately changed the connection status badge to an “OFFLINE” warning state. At the physical device level, the ESP32 microcontroller did not freeze but automatically activated offline mode; the sensor continued its sampling cycle and continuously pushed measurement data records along with standard timestamps into a circular buffer array dataBuffer stored in RAM.

After 30 seconds, Wi-Fi connectivity was restored. The control circuit immediately re-established the MQTT stream and entered the synchronization cycle. The buffer flush function buffer_pop was called continuously, pushing all trapped packets bearing past timestamps back to the EMQX Broker. At the Backend, the SQLite

database received and successfully verified the composite primary key, preserving the complete data sequence during the offline period. The clearest proof appeared when the user clicked the “Load History API” button on the Web Dashboard: the entire gap on the temperature-humidity chart was immediately filled in backfilled seamlessly, as if no network disconnection incident had occurred.



Figure 7.3: Data visualization chart before and after executing the Ring Buffer flush mechanism

7.4 Overall Evaluation

Comparing the results from the three experimental scenarios with the technical criteria set in Chapter 1, the prototype system demonstrated the validity of the design solution. The thermal exposure calculation algorithm combined with EMA filtering operated accurately at the edge, helping to filter out physical noise and prevent false alarms. The MQTT communication infrastructure combined with the offline ring buffer mechanism completely solved the issue of data loss due to network disruptions, preserving the integrity of historical time-series data. At the same time, the server layer and interface application Web Dashboard operated smoothly, responding instantly to alerts and successfully exporting CSV reports, fully achieving the goal of creating a smart and professional End-to-End cold-chain monitoring system.

Up to this point, all the core content of the thesis has been resolved! You only have Chapter 8 Conclusion and Future Work left to publish the final draft. Would you like me to go ahead and write a summary conclusion and introduce a few “future technology” ideas such as AI integration, Machine Learning, Blockchain data encryption, etc. so that the instructor sees the group's macro vision?

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Comprehensive Summary of Achievements

The project "Cold-Chain Monitoring and Intelligent Storage Management" has 100% completed all research objectives and technical requirements set forth. The system has been successfully verified through an End-to-End prototype model operating stably from the Edge Device Layer to the User Application Layer. Core achievements categorized by architectural tier include:

1. Achievements at Edge Devices (ESP32 Edge Computing):

Signal Processing & Noise Filtering: Successfully implemented the Exponential Moving Average EMA filtering algorithm with coefficient $\alpha = 0.2$ on the ESP32 microcontroller, completely eliminating ripple spike noise from the DHT22 sensor and ensuring an average measurement error of $MAE \leq 0.25^{\circ}\mathrm{C}$.

Thermal Exposure Calculation (Thermal FSM): Developed a Finite State Machine Thermal FSM executing real-time discrete Degree-Time integration. The system automatically classifies 3 operating states STATE_NORMAL, STATE_WARNING, STATE_VIOLATION, accurately quantifying the thermal exposure level when $T \notin [2.0^{\circ}\mathrm{C}, 8.0^{\circ}\mathrm{C}]$.

Network Fault Tolerance Mechanism: Implemented an offline circular buffer Offline Ring Buffer data structure with a capacity of 100 records in internal ESP32 RAM. In the event of Wi-Fi network disruptions, the system automatically buffers data and successfully backfills 100% of historical records as soon as connectivity is restored.

Non-blocking Actuator Control: Controlled the 28BYJ-48 stepper motor via the ULN2003 driver IC to rotate precisely 512 steps 90° simulating warehouse door operation, and activated a relay to turn on the 5V cooling fan when $T > 8.0^{\circ}\mathrm{C}$ without blocking the main execution loop.

2. Achievements at Server & Database (Backend & Database):

MQTT Ingestion & Processing: Built a Node.js/Express server connected via MQTT to the EMQX Broker broker.emqx.io on topic coldchain/nhom07/telemetry, asynchronously parsing JSON payload packets and automatically trapping $T \notin [2.0^{\circ}\mathrm{C}, 8.0^{\circ}\mathrm{C}]$ errors to log CRITICAL_TEMP incidents.

Time-Series Database Design (SQLite):

Configured the coldchain.db database consisting of two tables: telemetry_data and event_logs. Successfully applied a composite primary key PRIMARY KEY timestamp, device_id on the telemetry_data table, completely eliminating data duplication risks during buffer flushes from edge devices.

REST API Service:

Exposed HTTP API endpoints /api/history and /api/events standardized with a Response Envelope format and integrated with CORS flags, allowing client applications to retrieve the 50 most recent historical data points with ultra-low latency.

3. Achievements at User Interface (Web Dashboard Application):

Real-time Monitoring via WebSockets:

Built a Single-Page Application SPA using HTML5, Tailwind CSS, and MQTT.js, maintaining a direct WebSocket connection via port 8083 of EMQX.

Visual Alerts & Data Visualization:

Programmed dynamic temperature cards that automatically flash bright red animate-pulse upon safe range violations. Configured a dual-Y-axis Chart.js line graph Temperature in °C and Humidity in %RH displaying smooth real-time transitions.

Data Exploitation:

Successfully integrated the "🔄 Load History API" button to recover missing data gaps and the "📥 Export Report .CSV" button to download the complete telemetry log locally.

8.2 Scientific and Practical Contributions

1. Scientific & Methodological Contribution:

The project experimentally demonstrated the superiority of the Time-Aware Exposure Evaluation method compared to traditional Instantaneous Threshold approaches. By integrating the thermal exposure integral formula:

$$E = \sum_{i=1}^{N} \max(0, T_i - T_{\text{upper}}) \cdot \Delta t_i \quad \text{with } (T_{\text{upper}} = 8.0^{\circ}\text{C})$$

The system completely resolves the issue of false alarms. Short-term door openings < 60 seconds causing temporary air temperature spikes without altering the product's core temperature will not trigger severe violation alerts, reducing operational complacency among warehouse personnel.

2. Practical Value in Cold-Chain Logistics:

High Deployment Readiness:

The model uses open industrial-standard technologies MQTT, Node.js, SQLite, REST API, WebSockets with optimized hardware costs while delivering very high operational reliability.

Compliance Audit Trail Preservation:

Thanks to the local ring buffer mechanism and the backfill rendering capability on the Web Dashboard, the system ensures an uninterrupted compliance record. This is a prerequisite for pharmaceutical, medical, and agricultural export enterprises to meet strict audit standards such as WHO TRS 961, GDP Good Distribution Practices, and FDA CFR Part 11.

8.3 Technical Limitations

While achieving the intended objectives, the research team candidly identifies several existing technical limitations of the prototype model:

RAM-based Offline Buffer Capacity Limitations:

The dataBuffer ring buffer is allocated on the internal SRAM of the ESP32 with a fixed capacity of 100 records. At a sampling rate of 5000ms per record, the system can only maintain offline buffering for a maximum of 8.3 minutes during network outages. If a Wi-Fi disconnect lasts for several hours, older records will be overwritten, resulting in data loss beyond this threshold.

Communication Infrastructure & Security Limitations:

The system currently relies on the public broker broker.emqx.io over unencrypted TCP port 1883. The absence of Mutual TLS mTLS certificates and End-to-End data encryption exposes the system to eavesdropping or data injection risks from unauthorized devices on the public internet.

Single-Point Sensing Model Limitations:

The current model collects data from a single sensor location. In large-scale industrial cold storage facilities measuring thousands of cubic meters, air temperature distributes unevenly due to thermal boundary effects, stagnant air

circulation pockets, or proximity to cooling fans. A single sensor cannot accurately reflect the complete 3D spatial thermal profile of the entire facility.

Static Rule-Based Alert Mechanism:

The current FSM operates on fixed threshold boundaries 2.0°C - 8.0°C. The system lacks adaptive capabilities to automatically adjust parameters for different cargo types e.g., deep-freeze vaccines at -20°C, blood plasma, or fresh produce at 10°C - 15°C.

8.4 Innovative Future Directions

To advance the system from a prototype model toward an industrial-grade commercial product, the research team proposes five breakthrough directions:

1. Edge-AI / TinyML Integration for Early Incident Prediction:

Description:

Instead of issuing alerts only after the temperature exceeds 8.0°C, the team will embed a compressed deep learning model such as LSTM or Temporal Convolutional Network via TensorFlow Lite for Microcontrollers TFLite Micro directly onto the ESP32.

Breakthrough:

The AI model will analyze the time-series slope of temperature variation dT/dt and humidity. If an abnormal rate of temperature rise is detected even while T is still safe at 5.5°C, the AI will issue an early warning 15–30 minutes in advance predicting a refrigerant leak or compressor failure, enabling technicians to intervene before goods sustain damage.

2. Wireless Sensor Mesh Network Infrastructure (ESP-MESH) & 3D Thermal Mapping:

Description:

Expand from a single node to a self-organizing mesh sensor network ESP-MESH protocol consisting of 10–50 sensor nodes distributed across various heights and coordinates inside the warehouse.

Breakthrough: The network will compute the spatial variance index:

$$\Delta T_{\text{spatial}} = | T_{\max} - T_{\min} |$$

The Dashboard will construct a real-time 3D Thermal Heatmap, automatically highlighting “Hotspots” missed by cold air fan circulation, aiding in cargo stacking optimization.

3. Industrial-Grade Offline Storage Upgrade (SD-Card SPI Buffer with SQLite WAL Mode):

Description:

Replace the 100-record RAM buffer with a MicroSD card module communicating via high-speed SPI bus, integrated with a lightweight embedded SQLite library SQLite WAL - Write-Ahead Logging directly on the ESP32.

Breakthrough:

Expand offline buffering capacity up to 1,000,000 records equivalent to over 2 months of continuous storage during total Wi-Fi/Internet outages. The system guarantees zero data loss in long-distance cross-border cold-chain transport environments.

4. Blockchain Technology Application for Immutable Cold-Chain Auditing:

Description:

Combine mTLS/AES-256 encryption at the edge device with Smart Contract integration connected to a Blockchain network such as Hyperledger Fabric or Ethereum Testnet.

Breakthrough:

Every hour, the Backend server will hash SHA-256 the entire data sequence in the telemetry_data and event_logs tables to anchor it onto the Blockchain. This creates an “Immutable Compliance Audit Trail” that cannot be tampered with or modified to alter temperature logs, instilling absolute trust for international auditing bodies FDA/WHO and cargo insurance providers.

5. Ultra-Low-Power Sensor Node (Energy Harvesting) & LoRaWAN Communication:

Description:

Optimize ESP32 firmware with Deep Sleep mode, event-driven wake-up mechanisms, and thermoelectric generator TEG energy harvesting modules operating on ambient thermal gradients or warehouse lighting.

Breakthrough:

Replace short-range Wi-Fi connections with Low-Power Wide-Area Network LPWAN standards like LoRaWAN / NB-IoT. Sensor nodes will operate independently on battery and harvested energy continuously for 3–5 years without battery replacement, transmitting signals up to 10 km away, ready for installation on long-haul refrigerated container trucks.

REFERENCES

1. Espressif Systems, "ESP32 Series Datasheet," Version 4.1, 2024. [Online]. Available: https://documentation.espressif.com/esp32_datasheet_en.pdf

2. OASIS Open, "MQTT Version 5.0 Specification," OASIS Standard, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

3. World Health Organization (WHO), "Technical Report Series: Temperature and humidity monitoring systems for transport and storage of time- and temperature-sensitive pharmaceutical products," Annex 9, WHO, 2020. [Online]. Available: https://cdn.who.int/media/docs/default-source/medicines/norms-and-standards/guidelines/distribution/trs961-annex9-supp6.pdf

4. Chart.js, "Chart.js: Open source HTML5 Charts for most popular JavaScript framework," Version 4.4, 2024. [Online]. Available: https://www.chartjs.org/docs/latest/

5. SQLite Consortium, "SQLite Database Engine Documentation," Version 3.45, 2024. [Online]. Available: https://www.sqlite.org/docs.html

6. Aosong Electronics Co., Ltd., "Digital-output Relative Humidity and Temperature Sensor/Module DHT22 Datasheet," 2020. [Online]. Available: https://cdn.sparkfun.com/assets/f/7/d/9/c/DHT22.pdf

7. Bosch Sensortec, "BME280 Combined Humidity and Pressure Sensor Datasheet," Rev. 1.8, 2021. [Online]. Available: https://eu.mouser.com/en/ProductDetail/Bosch-Sensortec/BME280

8. Amazon Web Services (AWS), "FreeRTOS Kernel Developer Guide and Reference Manual," 2023. [Online]. Available: https://docs.aws.amazon.com/freertos/

9. Espressif Systems, "ESP-IDF Programming Guide," 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/

10. R. Fielding, M. Nottingham, and J. Reschke, "HTTP Semantics," IETF RFC 9110, Jun. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9110.html